

Week10_group_and_individual_assignment_Sulaiman (Stan) Abu Romman

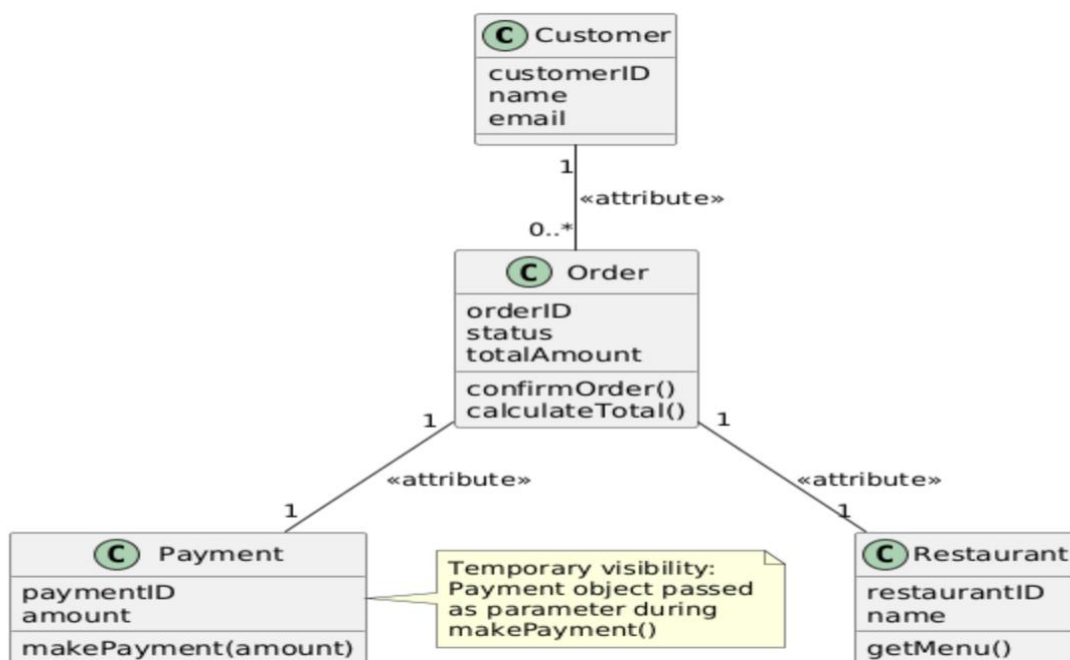
Part 1 – Group Assignment: Visibility (Design for Change)

Visibility refers to the ability of one object to know or access another object. In object-oriented design, visibility determines how objects communicate and how strongly they are coupled. Correct use of visibility helps reduce unnecessary dependencies and supports design for change.

In the Stomach Saver system, visibility is mainly achieved through attribute visibility and parameter visibility. Attribute visibility is used when objects need long-term access to each other, while parameter visibility is used when objects are temporarily passed during method execution. Global visibility is avoided to keep coupling low.

Expanded Design Class Diagram with Visibility – Stomach Saver

The following Design Class Diagram shows how visibility is applied in the Stomach Saver system. Associations represent attribute visibility, while method parameters represent temporary visibility.



Part 2 – Group Assignment: GRASP (Design for Change)

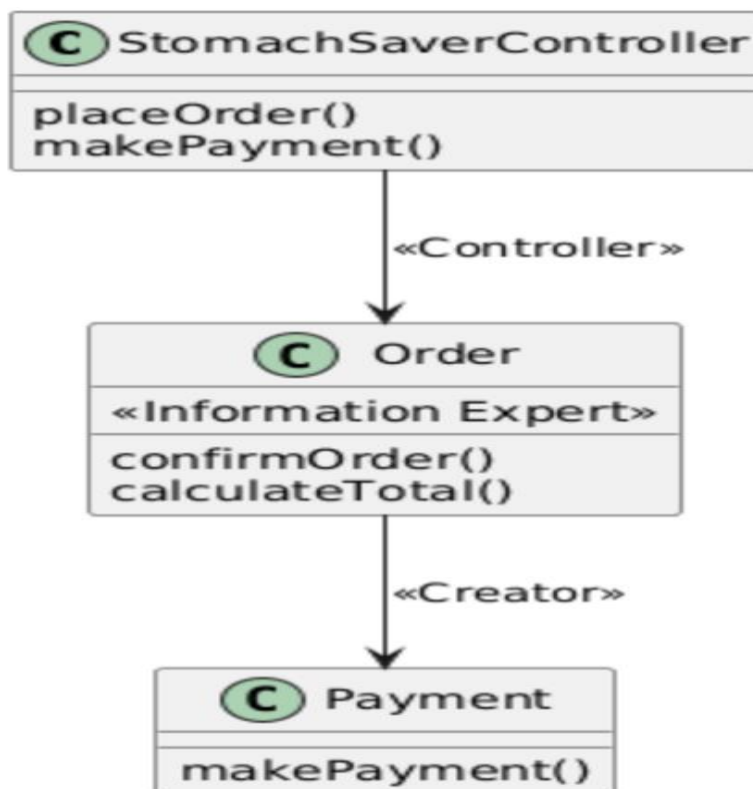
GRASP stands for General Responsibility Assignment Software Patterns. It provides principles for assigning responsibilities to classes in a way that results in low coupling and high cohesion.

In the Stomach Saver system, the Information Expert principle is applied by assigning the responsibility of calculating the total amount to the Order class, since it contains the required data. The Creator principle is applied by letting the Order class create Payment objects, as it closely manages and uses them.

The Controller pattern can be applied by introducing a controller that receives system events from the user interface and delegates responsibilities to domain classes such as Order and Payment. This prevents bloated domain objects and keeps responsibilities well separated.

Applying GRASP principles ensures that the Design Class Diagram remains flexible and supports future changes without major restructuring.

GRASP Applied to Stomach Saver – Visual Overview



Part 3 – Individual Assignment: Abstract Classes and Interfaces in a DCD

An abstract class is a class that cannot be instantiated and is used to define common behavior for its subclasses. It may contain both abstract and concrete methods. Abstract classes promote code reuse and shared structure.

An interface defines a set of methods that a class must implement without specifying how those methods are implemented. Interfaces are used to define roles and support flexible designs by reducing dependency on concrete classes.

In a Design Class Diagram, abstract classes are typically marked as abstract, and interfaces are shown using the «interface» stereotype. These techniques help separate general behavior from specific implementations.

The purpose of abstract classes and interfaces is to design systems that are easy to extend and modify. By relying on abstractions instead of concrete implementations, systems such as Stomach Saver can evolve without breaking existing functionality.

Visual representation – Abstract Class and Interface Example

